



Ollama con Python

En este taller aprenderás a usar la librería **Ollama Python** para interactuar con modelos de lenguaje locales desde tu código.

Verás cómo pasar de usar Ollama en la terminal a integrarlo en scripts y aplicaciones Python, para automatizar tareas, crear asistentes personalizados y experimentar con IA directamente desde tu entorno de desarrollo.

Instalar librería Ollama Python

¿Qué es la librería Ollama Python?

- Hasta ahora, probablemente has usado Ollama desde la terminal con comandos como `ollama run llama3`.
- La librería **Ollama Python** permite hacer lo mismo desde tu código, integrando el modelo en scripts y aplicaciones.

Del chat al código

- **Terminal (CLI):** conversación directa, ideal para pruebas rápidas.
- **Librería Python:** automatización, procesamiento de respuestas, integración en apps y pipelines.

Cuándo usar cada herramienta

¿Cuándo usar la terminal?

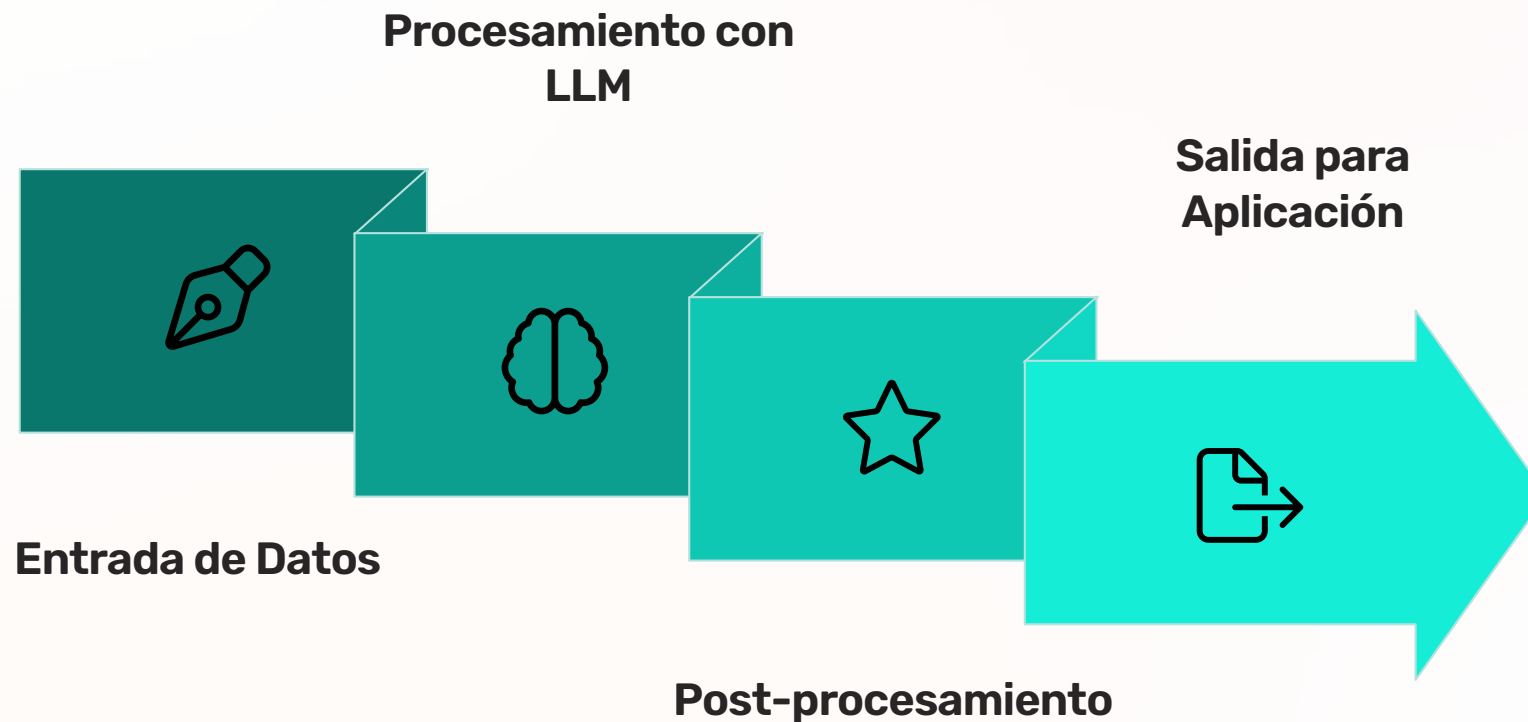
- Probar rápidamente un prompt o modelo nuevo.
- Obtener una respuesta puntual sin programar.
- Explorar capacidades de un modelo.

¿Cuándo usar la librería Python?

- Automatizar consultas repetitivas.
- Procesar múltiples preguntas o archivos.
- Integrar IA en proyectos más grandes.
- Guardar, analizar o transformar respuestas.

¿Qué es un Pipeline (Tubería) de IA?

Un "pipeline" es una secuencia de operaciones interconectadas que procesan datos de forma consecutiva. Cada paso transforma la información, llevando desde la entrada bruta hasta un resultado final utilizable.



Esta estructura es fundamental para integrar modelos de lenguaje grandes (LLM) como Ollama en aplicaciones, automatizando el flujo de trabajo para interactuar eficientemente con los modelos y procesar sus respuestas.

```
pythonlangton
ximalhíratatúle-antelette-lalmalutlx
pytr, pfseacik
doeiojze instenmoomx
et=er-ese inst.e..instanielor.llam.)
pt"-estook)
osatilil7ac: il=7llm*)
"echerelatea mel wettien.=lmslame.listkon.las)
?
4
&1
```

Prerrequisitos

01

Ollama instalado y funcionando

- Instalar desde ollama.com.
- Verificar con `ollama --version` y `ollama list`.

02

Python 3.8+ instalado

- Verificar con `python --version` o `python3 --version`.

03

Al menos un modelo descargado

- Ejemplos: `ollama pull llama3` o `ollama pull mistral`.

Instalación y primer script

Instalación de la librería Ollama Python

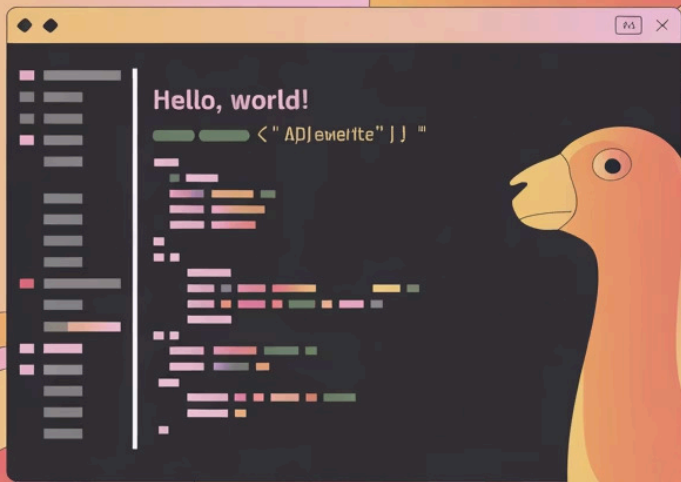
```
pip install ollama
```

o

```
pip3 install ollama
```

Verificar que Ollama está corriendo

- Ejecutar `ollama list` en la terminal.
- Si ves lista de modelos (aunque esté vacía), Ollama está activo.



OLAMA

Hola Mundo con Ollama

Primer programa: "Hola Mundo"

- Importar la librería ollama.
- Usar `ollama.chat()` con un mensaje de usuario.
- Acceder a la respuesta en `response['message']['content']`.

📄 Este ejemplo funciona tanto en un archivo .py como en un notebook (.ipynb).

Chat con Ollama: conceptos básicos

Función clave: chat()

Es la forma principal de interactuar con modelos y mantener contexto.

Estructura de mensajes

role:

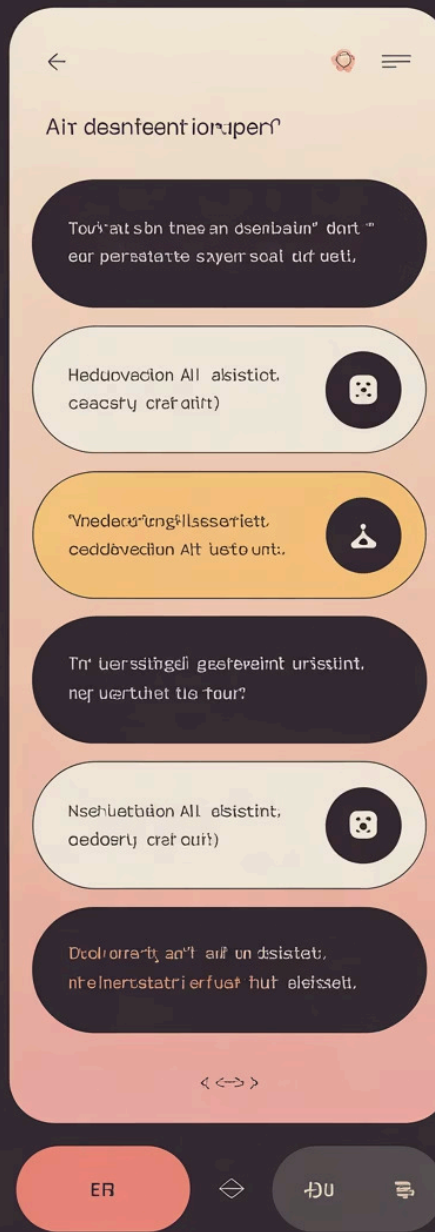
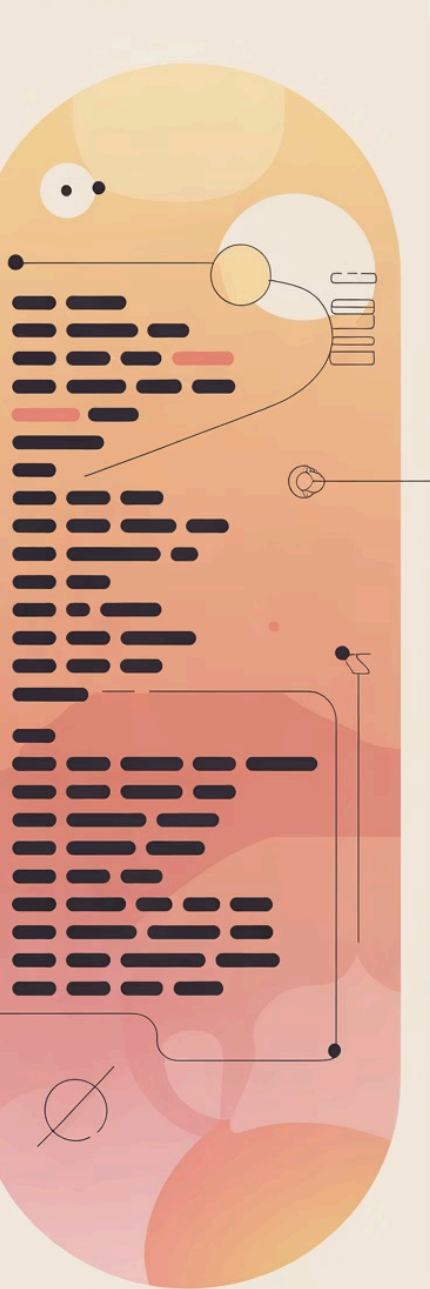
- **user** → mensajes de la persona.
- **assistant** → respuestas previas del modelo.
- **system** → instrucciones especiales.

content: contenido del mensaje (texto).

Chat básico: un mensaje, una respuesta

- Caso de uso: preguntas independientes sin necesidad de contexto previo.
- Se pasa una lista con un solo mensaje user.
- Se imprime directamente la respuesta de chat().

Este patrón es útil para consultas simples, tests rápidos y scripts sin memoria de conversación.



Ejemplos prácticos de chat

Ejemplo: Asistente explicador de código

- System prompt: "explica código línea por línea de forma didáctica".
- Se le pasa un bloque de código Python como content.
- El modelo devuelve una explicación detallada.

chat() vs generate()

Diferencias principales

chat()

- Pensado para conversaciones con contexto.
- Soporta roles, historial y system prompts.
- Ideal para asistentes, chatbots y flujos interactivos.

generate()

- Generación de texto simple, sin contexto conversacional.
- Útil para completar un prompt único.

En este taller el foco está en **chat()** por su versatilidad.

Streaming: ver la respuesta en tiempo real

¿Qué es el streaming?

- Permite recibir la respuesta por fragmentos mientras se genera.
- Mejora la experiencia de usuario, especialmente con respuestas largas.

Uso básico

- Añadir `stream=True` a la llamada a `chat()`.
 - Iterar sobre los chunk y mostrar `chunk['message']['content']` según lleguen.
-

Cuándo no usar streaming

- Cuando necesitas procesar la respuesta completa antes de seguir.
- Cuando las respuestas son cortas.

Trabajar con JSON

Por qué JSON

- Formato estándar, fácil de parsear en Python.
- Permite convertir respuestas del modelo en diccionarios y listas.

Solicitar JSON al modelo

- Pedir explícitamente que responda en JSON.
- Usar `options={'format': 'json'}` en `chat()`.
- Parsear con `json.loads(response['message']['content'])`.

Ejemplos prácticos con JSON

1

Extracción de información

- Pedir datos estructurados (nombre, edad, ciudad, trabajo, etc.) a partir de un texto.
- Diseñar un system prompt que defina claramente las claves y el tipo de datos.

2

Procesado por lotes

- Iterar sobre una lista de textos.
- Para cada uno, llamar al modelo con `format: 'json'`.
- Parsear y guardar los resultados en una lista de diccionarios.

3

Buenas prácticas

- Especificar formato y claves con detalle.
- Incluir ejemplos de salida JSON.
- Manejar errores de parseo con `try/except`.

Librería OpenAI Python con Ollama

Por qué usar la librería OpenAI con Ollama

- API muy conocida y ampliamente documentada.
- Permite cambiar solo la configuración del cliente para pasar de local a cloud.

Cuándo elegir cada opción

Librería Ollama nativa

Simplicidad si solo usas Ollama.

Librería OpenAI

Portabilidad y flexibilidad entre diferentes IAs.

OpenAI Python + Ollama (local)

Configuración básica

- `base_url="http://localhost:11434/v1"`
- `api_key="ollama"` (cualquier string sirve, no hay auth real).